Ch. 10 Notes
Graphs

Vocabulary

- **Vertex** (V), **edge** (E). $n = |V|$, $m = |E|$.
- **Directed** (arcs) vs. **undirected**.
- **Weighted**: edges carry a weight $w(u, v)$.
- **Path / cycle**. **DAG**: directed acyclic.
- **Connected**: every pair reachable (undirected).
- **Sparse**: $m = O(n)$. **Dense**: $m = \Theta(n^2)$.

Representations

	adj. list	adj. matrix
space	$O(n+m)$	$O(n^2)$
has-edge?	$O(\deg(u))$	$O(1)$
iterate nbrs	$O(\deg(u))$	$O(n)$
best for	sparse	dense, heavy has-edge queries

```
using AdjList = std::vector<std::vector<
    int>>;
// weighted:
using W = std::vector<std::vector<std::
    pair<int,int>>>;
```

BFS (unweighted shortest path)

Queue, visited set, parent / dist arrays.

```
std::queue<int> q; q.push(s);
dist[s] = 0;
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (dist[v] ==
        -1) {
        dist[v] = dist[u] + 1;
        parent[v] = u;
        q.push(v);
    }
}
```

$O(n + m)$. Gives shortest path in *edge count*.

DFS

Stack (iterative) or recursion. Uses: cycle detection, topo sort, connectivity, tarjan's SCC. $O(n + m)$.

BFS vs. DFS

pick BFS when	pick DFS when
shortest in # edges	topo / SCC / cycles
level-by-level	all paths / backtracking
small branching	deep graphs

Dijkstra (non-negative weights)

Min-heap of `(dist, vertex)`. On pop, skip if stale.

```
pq.push({0, s}); dist[s] = 0;
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;    //
        lazy delete
    for (auto [v, w] : adj[u])
        if (d + w < dist[v]) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
}
```

Heap version: $O((n + m) \log n)$. Breaks on negative edges.

Bellman-Ford (negative weights OK)

Relax every edge $V - 1$ times. One more pass detects a negative cycle.

$$O(V \cdot E).$$

Use when some edges are negative (currency arbitrage, reward nets).

Topological sort (DAG only)

Kahn (BFS of in-degree 0):

1. Compute in-degree of every vertex.
2. Push all zero-in-degree into a queue.
3. Pop, emit, decrement in-degree of neighbors, push new zeros.

If emitted count $< n$, a cycle exists.
DFS version: post-order push onto a stack; reverse is topo. Both $O(n + m)$.

Minimum spanning tree

Kruskal: sort edges, take if endpoints in different components (use union-find). $O(m \log m)$.
Prim: grow a single tree; min-heap of crossing edges. $O(m \log n)$. Prefer Prim on dense, Kruskal on sparse.

Floyd-Warshall (all-pairs)

```
for (int k = 0; k < n; ++k)
  for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
      if (d[i][k] + d[k][j] < d[i][j])
        d[i][j] = d[i][k] + d[k][j];
```

$O(n^3)$, $O(n^2)$ space. Handles negative edges (no negative cycles). Beats $V\times$ Dijkstra once $m \approx n^2$.

Algorithm picker

task	algo
reachability, unweighted	BFS / DFS
shortest path, unweighted	BFS
shortest path, $w \geq 0$	Dijkstra
shortest path, any w	Bellman-Ford
all-pairs shortest paths	Floyd-Warshall
schedule DAG	topo sort
minimum spanning tree	Kruskal / Prim
strong connectivity	Tarjan / Kosaraju (DFS)

Top gotchas

- Running Dijkstra on a graph with negative edges – silently wrong, not an error.
- Forgetting the *lazy-delete* check in Dijkstra's PQ (`if (d > dist[u]) continue;`) – blows up runtime.
- Topo sort on a graph with cycles – Kahn emits fewer than n nodes; handle that case.
- Using an adj matrix on a sparse graph – $O(n^2)$ space for nothing.
- Writing BFS with a stack, or DFS with a queue. Check data structure before coding.